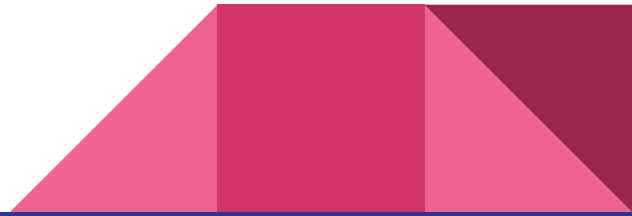
SPECTRAL CLUSTERING

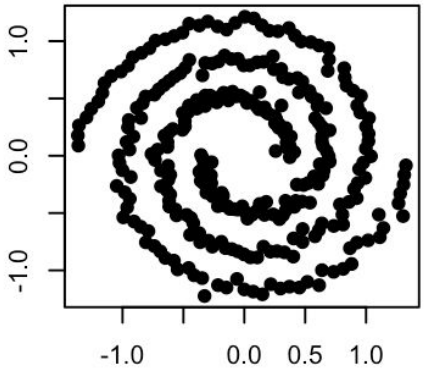
- Akash 22PD03
- Varun Sithaarth KK 22PD40

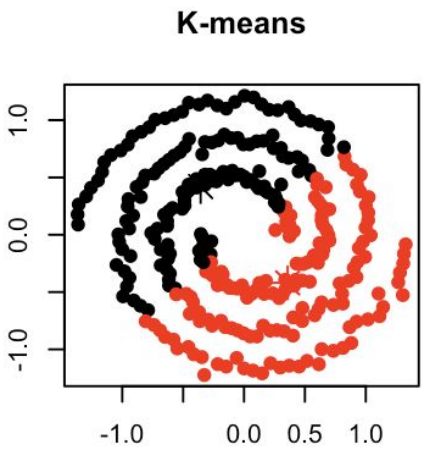
INTRODUCTION

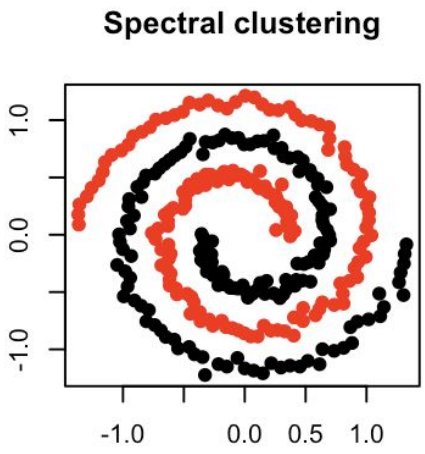
- In spectral clustering, data points are treated as nodes of a graph. Thus, spectral clustering is a graph partitioning problem.
- The nodes are then mapped to a low-dimensional space that can be easily segregated to form clusters.
- No assumption is made about the shape/form of the clusters.
- The goal of spectral clustering is to cluster data that is connected but not necessarily compact or clustered within convex boundaries.

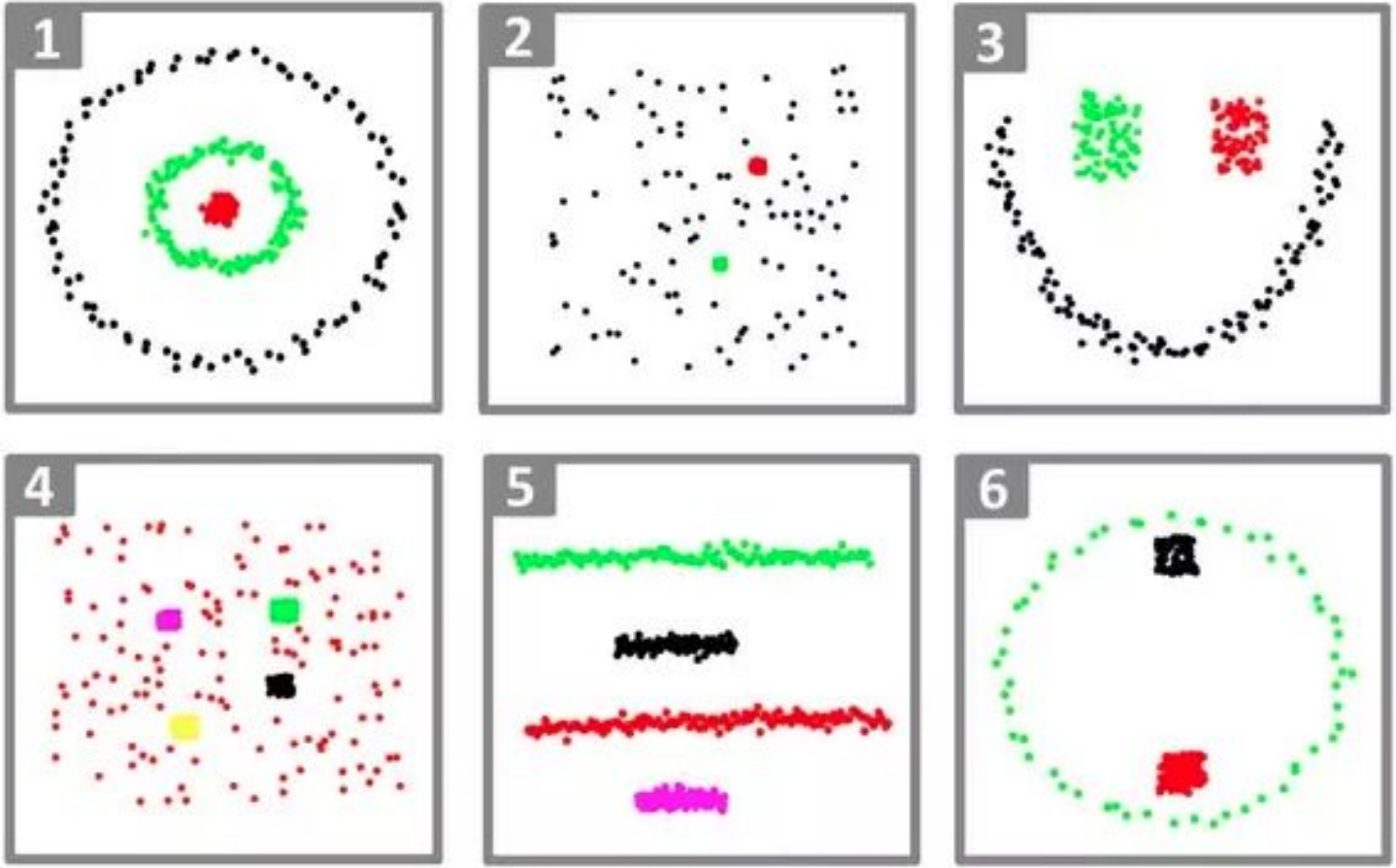


SPECTRAL vs K-MEANS CLUSTERING







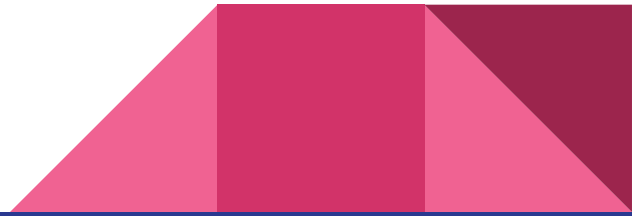

1
2
3
4
5
6

APPLICATIONS:

- Image processing
- Pattern recognition
- Protein Sequencing
- Statistical Data analysis
- Graph Partitioning

ALGORITHM

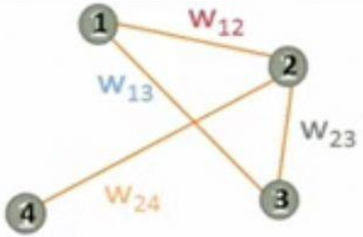
- Define a similarity matrix S.
- Construct the Graph Laplacian from L = D - S, where D is a diagonal matrix with d_{ii}=sum of i^{th} row in S.
- Solve the Eigenvalue problem for L.
- Select k eigenvectors corresponding to the k lowest (or highest) eigenvalues to define a k-dimensional subspace
- Form clusters in this subspace using k-means.



Matrix Representations of a Graph

- Adjacency Matrix: n x n symmetric matrix

$$A_{ij} = \begin{cases} w_{ij} & : \text{weight of edge } (i,j) \\ 0 & : \text{if no edge between } i,j \end{cases}$$



- The Laplacian

$$L = D - A \qquad d_i = \sum_{\{j|(i,j)\in E\}} w_{ij}$$

where D is the diagonal matrix of degrees

$$L_{ij} = \begin{cases} d_i & : \text{if } i = j \\ -w_{ij} & : \text{if } (i,j) \text{ is an edge} \\ 0 & : \text{if no edge between } i,j \end{cases}$$

$d_1 = w_{12} + w_{13}$
$d_2 = w_{12} + w_{23} + w_{24}$
$d_3 = w_{13} + w_{23}$
$d_4 = w_{24}$

A =

	1	2	3	4
1	0	w_{12}	w_{13}	0
2	w_{12}	0	w_{23}	w_{24}
3	w_{13}	w_{23}	0	0
4	0	w_{24}	0	0

L =

	1	2	3	4
1	d_1	$-w_{12}$	$-w_{13}$	0
2	$-w_{12}$	d_2	$-w_{23}$	$-w_{24}$
3	$-w_{13}$	$-w_{23}$	d_3	0
4	0	$-w_{24}$	0	d_4

Eigenvalue and Eigenvector of Graphs

- For a matrix **A**, λ is an *eigenvalue* of **A** if for some vector ***v***,

$$A\nu = \lambda\nu$$

v is the *eigenvector* of **A** corresponding to λ.

- For a graph G with *n* nodes, its adjacency has *n* eigenvalues $\{\mu_1, \mu_2, \ldots, \mu_n\}$ where $\mu_1 \geq \mu_2 \geq \ldots \geq \mu_n$, and *n* corresponding eigenvectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n\}$

Spectrum of a graph:
$\mu_1 \geq \mu_2 \geq \ldots \geq \mu_n$

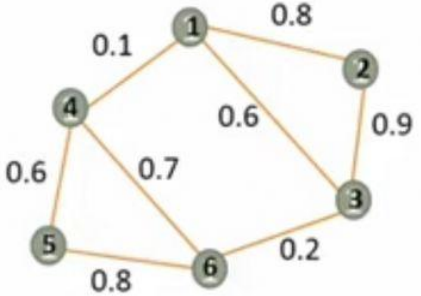


Figure 1. Graph G with adjacency matrix **A**

A =

	1	2	3	4	5	6
1	0	0.8	0.6	0.1	0	0
2	0.8	0	0.9	0	0	0
3	0.6	0.9	0	0	0	0.2
4	0.1	0	0	0	0.6	0.7
5	0	0	0	0.6	0	0.8
6	0	0	0.2	0.7	0.8	0

$\mu_1 = -0.993$	$\boldsymbol{x}_1 = [-0.17, 0.65, -0.56, -0.11, -0.25, 0.39]$
$\mu_2 = -0.832$	$\boldsymbol{x}_2 = [-0.46, 0.44, 0.00, 0.30, 0.37, -0.61]$
$\mu_3 = -0.642$	$\boldsymbol{x}_3 = [-0.37, -0.03, 0.36, 0.53, -0.66, 0.13]$
$\mu_4 = -0.482$	$\boldsymbol{x}_4 = [-0.57, 0.06, 0.48, -0.56, 0.19, 0.31]$
$\mu_5 = 1.355$	$\boldsymbol{x}_5 = [0.25, 0.30, 0.24, -0.48, -0.52, -0.52]$
$\mu_6 = 1.584$	$\boldsymbol{x}_6 = [-0.48, -0.53, -0.52, -0.26, -0.25, -0.30]$

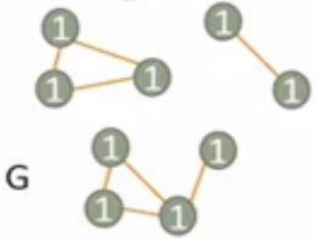
Eigenvalues and eigenvectors of A

Eigenvalue and Eigenvector of Graphs

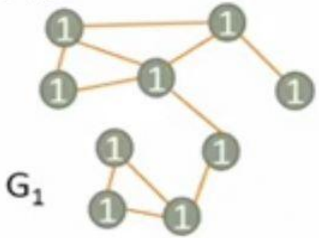
- The graph Laplacian of G, L_G, also has
  - eigenvalues $\{\lambda_1, \lambda_2, \ldots, \lambda_n\}$ where $0 = \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$, *and*
  - eigenvectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$

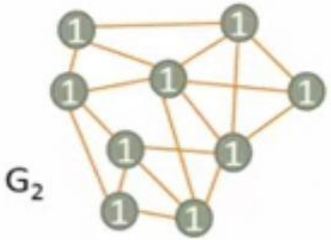
Spectrum of the Laplacian:
$0 = \lambda_1 \leq \lambda_2 \leq \ldots \leq \lambda_n$

- Eigenvalues reveal global graph properties not apparent from edge structure
  - If 0 is the eigenvalue of L with *k* different eigenvectors, i.e., $0 = \lambda_1 = \lambda_2 \ldots = \lambda_k$, then G has *k* connected components
  - If the graph is connected, $\lambda_2 > 0$ and λ_2 is the **algebraic connectivity** of G
    - The greater λ_2, the more connected G is



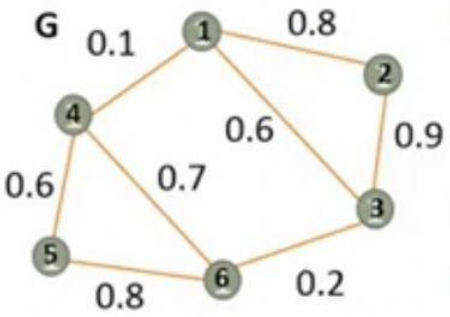
L_G has $0 = \lambda_1 = \lambda_2 = \lambda_3$ and $\lambda_4 > 0$





Both L_{G1} and L_{G2} have $0 = \lambda_1$ and $\lambda_2 > 0$. $\lambda_2(L_{G1}) < \lambda_2(L_{G2})$

Bipartitioning via Spectral Methods



$\lambda_2(L_G) = 0.189$

- To find the bipartition, we take the second eigenvector of the Laplacian, $\mathbf{v}_2$, corresponding to λ_2, the algebraic connectivity of G.
  - The smaller λ_2, the better quality of the partitioning
- For each node *i* in G, assign it the value $\mathbf{v}_2(i)$ e.g., $\mathbf{v}_2(1) = 0.41$ in G
- To find clusters C_1 and C_2, assign nodes with $\mathbf{v}_2(i) > 0$ to C_1 and $\mathbf{v}_2(i) < 0$ to C_2.

$C_1 = \{ i \mid \mathbf{v}_2(i) > 0 \}$,
$C_2 = \{ i \mid \mathbf{v}_2(i) < 0 \}$

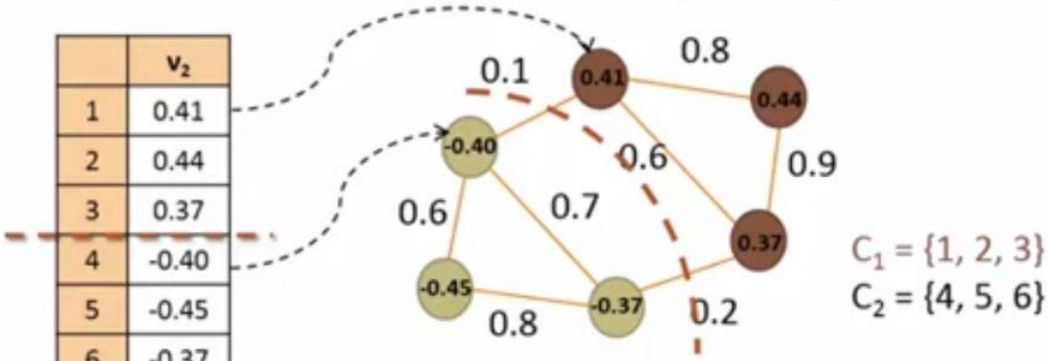
$L_G =$

	1	2	3	4	5	6
1	1.5	-0.8	-0.6	-0.1	0	0
2	-0.8	1.7	-0.9	0	0	0
3	-0.6	-0.9	1.7	0	0	-0.2
4	-0.1	0	0	1.4	-0.6	-0.7
5	0	0	0	-0.6	1.4	-0.8
6	0	0	-0.2	-0.7	-0.8	1.7

Laplacian of **G**

	$\mathbf{v}_2$
1	0.41
2	0.44
3	0.37
4	-0.40
5	-0.45
6	-0.37

Second eigenvector of L_G



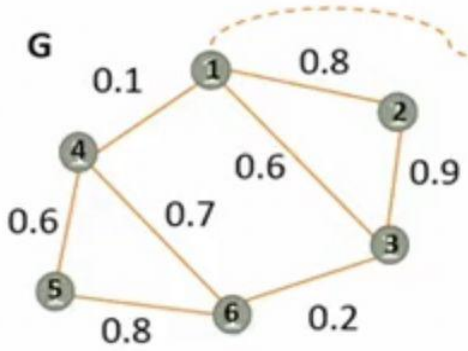
Nodes in **G** mapped onto $\mathbf{v}_2$.
Bipartition of G based $\mathbf{v}_2$

Extension to *k* partitions

- **The Ng-Jordan-Weiss (NJW) Algorithm** (2002) $L_{norm} = D^{-1/2} L D^{-1/2}$
  - Compute the first k eigenvectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ of L_{norm}, the normalized Laplacian
  - Let $U \in R^{n \times k}$ be the matrix containing the vectors $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k$ as columns
  - For $i = 1, \ldots, n$, take the i-th row of U as its feature vector after normalizing to norm 1
  - Cluster the points with k-means into k clusters $C_1, \ldots, C_k$
  - Commonly used as a dimension reduction technique for clustering

	1	2	3	4	5	6
1	1.0	-0.5	-0.4	-0.1	0	0
2	-0.5	1.0	-0.5	0	0	0
3	-0.4	-0.5	1.0	0	0	-0.1
4	-0.1	0	0	1.0	-0.4	-0.5
5	0	0	0	-0.4	1.0	-0.5
6	0	0	-0.1	-0.5	-0.5	1.0

$L_{norm}(G)$



	v_1	v_2	v_3
1	$v_1(1)$	$v_2(1)$	$v_3(1)$
2	$v_1(2)$	$v_2(2)$	$v_3(2)$
3	$v_1(3)$	$v_2(3)$	$v_3(3)$
4	$v_1(4)$	$v_2(4)$	$v_3(4)$
5	$v_1(5)$	$v_2(5)$	$v_3(5)$
6	$v_1(6)$	$v_2(6)$	$v_3(6)$

U for $k = 3$

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.preprocessing import StandardScaler, normalize
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
```

In [2]:

```
raw_df = pd.read_csv('../input/ccdata/CC GENERAL.csv')
raw_df = raw_df.drop('CUST_ID', axis = 1)
raw_df.fillna(method ='ffill', inplace = True)
raw_df.head(2)
```

Out[2]:

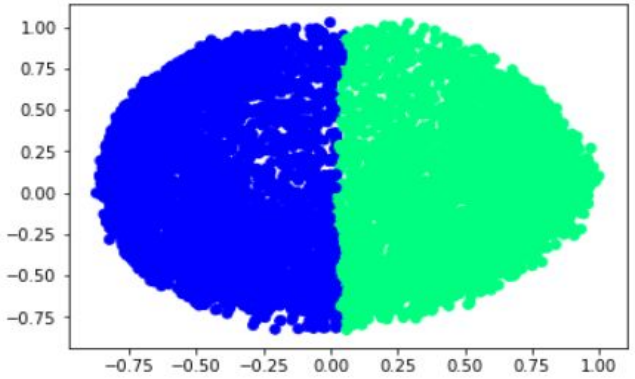
	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENC
0	40.900749	0.818182	95.4	0.0	95.4	0.000000	0.166667
1	3202.467416	0.909091	0.0	0.0	0.0	6442.945483	0.000000

In [3]:

```
# Preprocessing the data to make it visualizable

# Scaling the Data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(raw_df)

# Normalizing the Data
X_normalized = normalize(X_scaled)

# Converting the numpy array into a pandas DataFrame
X_normalized = pd.DataFrame(X_normalized)

# Reducing the dimensions of the data
pca = PCA(n_components = 2)
X_principal = pca.fit_transform(X_normalized)
X_principal = pd.DataFrame(X_principal)
X_principal.columns = ['P1', 'P2']

X_principal.head(2)
```

In [4]:
```
# Building the clustering model
spectral_model_rbf = SpectralClustering(n_clusters = 2, affinity ='rbf')

# Training the model and Storing the predicted cluster labels
labels_rbf = spectral_model_rbf.fit_predict(X_principal)
```

In [5]:
```
# Visualizing the clustering
plt.scatter(X_principal['P1'], X_principal['P2'],
           c = SpectralClustering(n_clusters = 2, affinity ='rbf') .fit_predict(X_principal), cmap =plt.cm.winter)
plt.show()
```

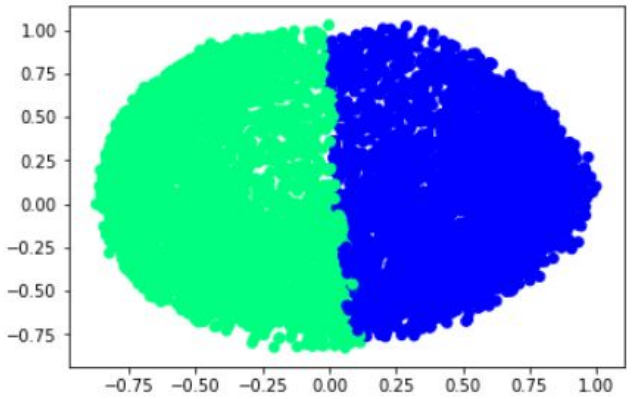


In [6]:

```
# Building the clustering model
spectral_model_nn = SpectralClustering(n_clusters = 2, affinity ='nearest_neighbors')

# Training the model and Storing the predicted cluster labels
labels_nn = spectral_model_nn.fit_predict(X_principal)
```

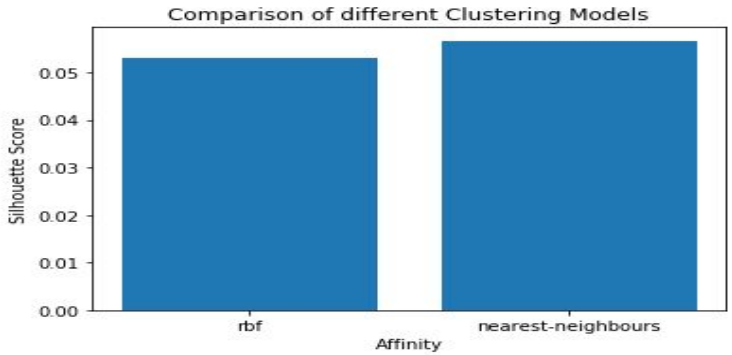
In [7]:

```
# Visualizing the clustering
plt.scatter(X_principal['P1'], X_principal['P2'],
           c = SpectralClustering(n_clusters = 2, affinity ='nearest_neighbors') .fit_predict(X_principal), cmap =plt.cm.wi
nter)
plt.show()
```



In [8]:

```
# List of different values of affinity
affinity = ['rbf', 'nearest-neighbours']

# List of Silhouette Scores
s_scores = []

# Evaluating the performance
s_scores.append(silhouette_score(raw_df, labels_rbf))
s_scores.append(silhouette_score(raw_df, labels_nn))

# Plotting a Bar Graph to compare the models
plt.bar(affinity, s_scores)
plt.xlabel('Affinity')
plt.ylabel('Silhouette Score')
plt.title('Comparison of different Clustering Models')
plt.show()

print(s_scores)
```




```
[0.05300611480757429, 0.05667039590382262]
```

PROS AND CONS

PROS :

- No assumptions are made in regards to shape of the data. In k-means clustering we assume the shape of every cluster to be spherical.
- We don’t spend a lot of iterations calculating centroid of a cluster.
- It can accommodate data with varying shapes and sizes.
- It is relatively a low cost algorithm upto an input size of a few thousand data points.

PROS AND CONS

CONS :

- Computing eigenvalues and eigenvectors and then clustering them is costly for large datasets.

THANK YOU